Chapter route

How does a UNIX-influenced kernel compose processes, resources, and modular interfaces?

Prerequisites are processes/threads, virtual memory, filesystems, and modules. This case fixes Linux v3.2 rather than describing unchecked current behavior.

- Distinguish historical design from present systems.
- Trace fork/exec, module loading, and page-cache paths.
- Analyze historical CFS/predecessor, reclaim, and filesystem boundaries.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Kernel, libraries, and distributions

Linux v3.2 provides tasks, VM, VFS, IPC, networking, and drivers. GNU tools, shell, libc, and package managers mostly run in user space. A distribution integrates kernel, tools, configuration, installation, and updates; its name does not specify every CONFIG option.

Two teaching machines share v3.2/x86. An ELF requests PT_INTERP=/lib/ld-linux.so.2. A provides it;B does not. Both find the executable, but B fails obtaining the interpreter. Kernel-version equality is insufficient. Static/dynamic linking, ISA, ABI, and library symbol versions impose different compatibility conditions.

Component	Responsibility	Observable failure
Kernel image/CONFIG	Privileged syscalls/device support	Missing driver prevents device use
ELF interpreter/libc	Usually user-mode loading/API wrappers	Missing interpreter fails exec;missing symbol fails linking
Shell/utilities	User processes using services	Missing command does not remove fork/read
Distribution policy	Packages/default integration	Different permissions/startup behavior

Kernel, libraries, and distributions (continued)

From its 1991 beginnings, Linux expanded to SMP and many architectures. Modules, fine-grained locks, and standard interfaces address hardware diversity, multitasking, and compatibility. Concrete claims still depend on versioned source/configuration.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Protection boundaries inside a modular kernel

Scheduler, memory, VFS, and most drivers share one kernel protection domain. Loading independently does not create isolated user servers. Ordinary function calls cross module boundaries without necessarily switching address spaces.

For one synchronous buffered read, syscall enters kernel, VFS calls filesystem, then block/driver code. User/kernel crossings change mode;internal calls do not. Waiting can schedule another task, adding a distinct context/address-space switch. No universal read switch count follows.

A bad driver pointer can corrupt kernel data. ABI/type/signature checks do not create another isolation domain. A user-space file service can use page protection but requires IPC, data-transfer, and server-failure protocols. Chapter 2 compares fixed paths;this chapter connects Linux objects.

UNIX/POSIX APIs specify observable fd/process/signal semantics, not a unique internal structure. Read can serve disk, pipe, socket with different seeking, blocking, and EOF contracts.

Module loading as a resource transaction

The following is a custom publish-last driver protocol, not a visibility guarantee for all Linux 3.2 registrations. Driver M imports A/B and registers D. Validate format/version, allocate/relocate, resolve exports, initialize, then publish. Finding A without B cannot expose a callable half-built D. If init allocates a buffer then registration fails, unwind successful acquisitions in reverse order.

Stage	Visibility	Resources	Failure action
Before resolution	Not generally callable	Load region	Reject validation failure
Resolve A/B	Not LIVE	Relocated code/data	Release on unresolved import
Init	Service being built	Queues and buffers; registration not yet published	Undo completed acquisitions
LIVE	New calls retain references	Registered service	Enforce per-call lifetime

In Linux 3.2 some registered entry points may be called before init returns. Drivers must initialize externally visible state before registration, then revoke entry points and synchronize concurrent calls on init failure. The table's init row describes only private preparation under this example's publish-last protocol.

Zero references do not close entry points

This driver assigns one M reference per handle/request. Initially three handles plus two requests give 5.

Stage	New references?	Handles	Requests	References	May free code?
LIVE	Yes	3	2	5	No
GOING, entries withdrawn	No	3	2	5	No
Handles closed	No	0	2	2	No
I/O finished/legally canceled	No	0	0	0	Callbacks may remain
All callbacks/workers quiescent	No	0	0	0	Yes

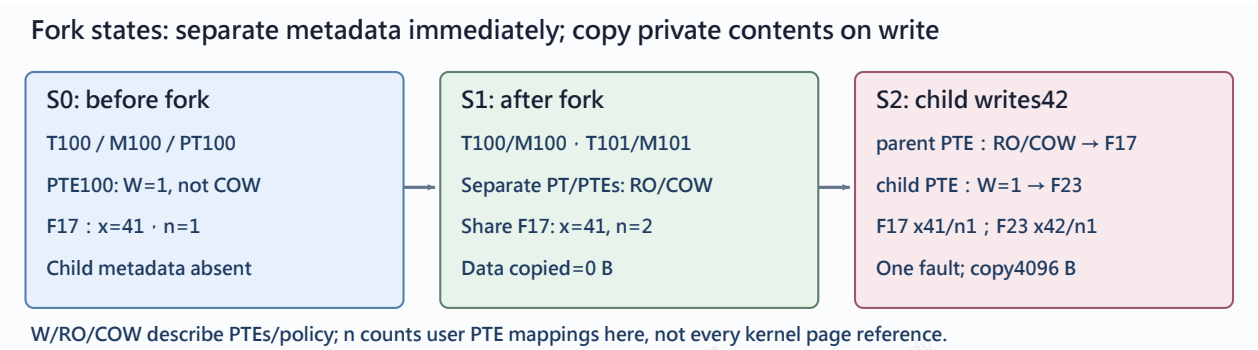
Checking zero before stopping entry lets another CPU retain and execute code about to be freed. State/registration protocol first prevents entry, then drains use. RCU callbacks need their grace periods;workqueues, timers, DMA need their own synchronous cancellation/quiescence. One RCU wait is not sufficient for every module.

Four layers of fork state

In v3.2, copy_process/dup_task_struct allocates child task/stack, then initializes PID, credentials, scheduler, and other execution context. Ordinary fork uses copy_mm/dup_mm for independent mm_struct/page tables. Copy_page_range may map shared private frames write-protected. Some VMA page-table entries can be established lazily by a later fault. A write fault copies only when separation is required; a safely exclusive anonymous page may be reused and made writable. There is no wait-until-first-metadata-write allocation of task_struct.

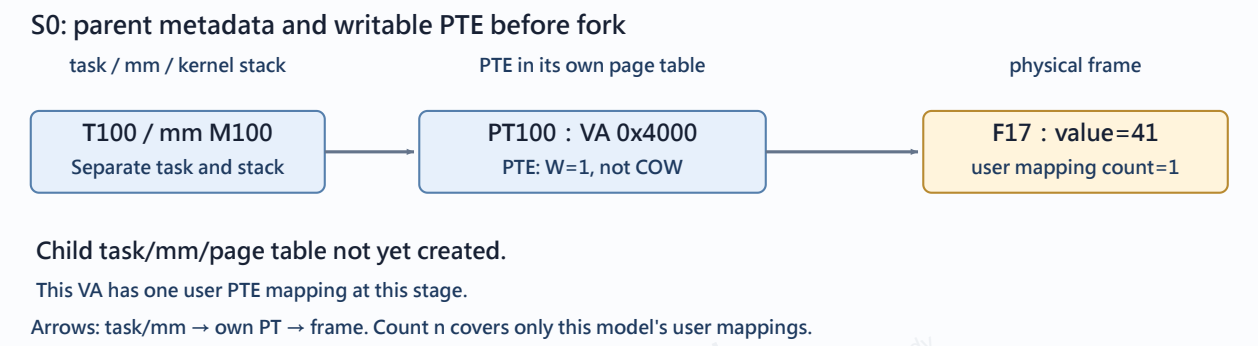
Layer	Ordinary fork	First private write
Task/kernel stack	Separate PID, state, stack objects	Not deferred by data COW
Mm/page tables	Separate objects	Child updates its PTE
Eligible resident private anonymous page	May share a temporarily write-protected frame	Copies if separation is needed; an exclusive page may be reused
Open-file descriptions	Existing descriptors can share objects	Offset sharing is unrelated to user-page COW

Four layers of fork state (continued)



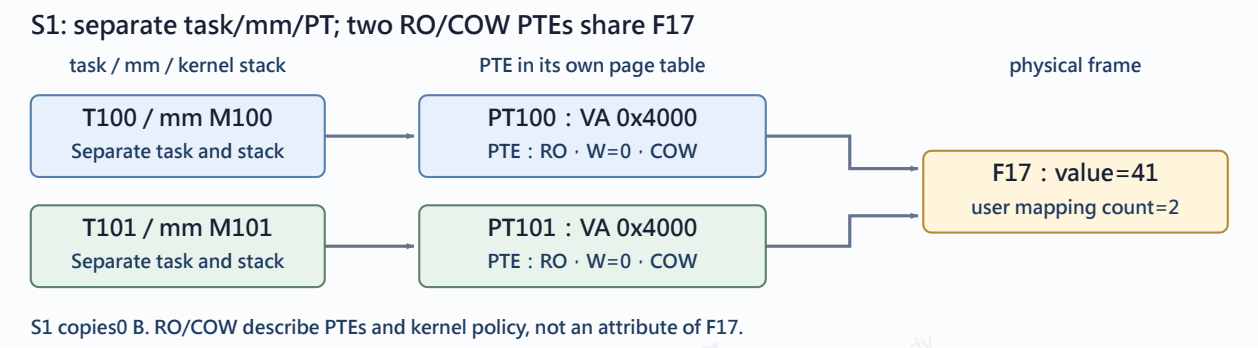
Clone parameterizes sharing. CLONE_VM shares mm;CLONE_FILES descriptor table;CLONE_FS filesystem context;CLONE_SIGHAND handler table. Thread-like sharing does not make ordinary fork share task metadata.

S0 before fork



T100/M100/PT100 maps VA 0x4000 to frame 17, x=41, W=1, user-PTE mapping count 1. Child task does not exist. This is the baseline, not a preexisting COW state.

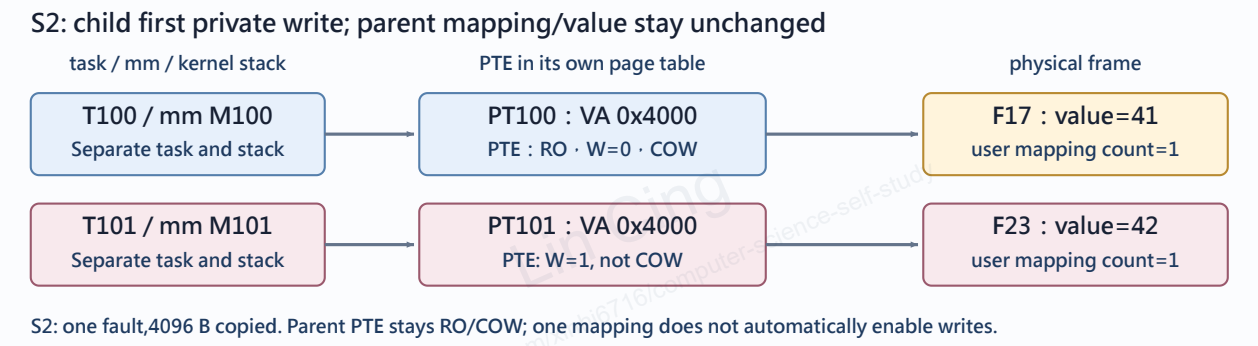
S1 separate metadata, shared frame



Child T101/M101/PT101 already has separate PID, state, kernel stack, mm, page table. Both PTEs map F17 with writes disabled under COW policy. F17 remains 41, mapping count 2, data copied 0 bytes.

S2 first child write

Assume one 4 KiB anonymous private page, logically writable VMA, x at offset 0 of VA 0x4000, initial 41. Exclude huge pages, other mappings, DMA, concurrent writes. COW is a policy label, not an x86 hardware PTE bit named COW. Counts cover this model's user PTE mappings rather than all Linux page references.



S2 first child write (continued)

State	Parent	Child	Frame contents/mappings
S0	T100/M100/PT100:F17, W1	Absent	F17:x41, n1
S1	T100/M100/PT100:F17, W0, COW	T101/M101/PT101:F17, W0, COW	F17:x41, n2;copied0
S2 writes 42	Parent unchanged	T101/M101/PT101:F23, W1	F17:x41, n1;F23:x42, n1;one fault,4096 B copied

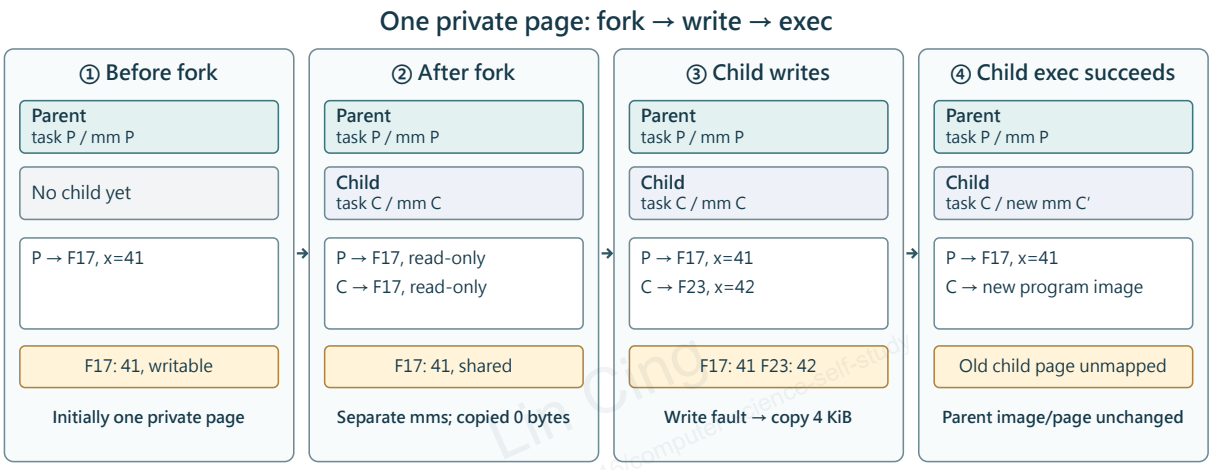
Observable counterexamples and CLONE_VM

If child and parent task were one object, setting child PID 101 would change parent PID too. Correctly, child ready→blocked leaves parent ready. Shared data pages do not imply shared tasks.

Successful exec in this single-thread child preserves process PID while replacing its program image. Early failure before destroying the old image can return an error. After v3.2 flush_old_exec crosses the point of no return, some later failures terminate by signal rather than restore the old image.

CLONE_VM is a different explicit option:two tasks share mm/tables. With writable F17, child writes 42, parent sees 42 without this private-COW fault. Ordinary fork instead yields parent 41/child 42. These finite models expose differences rather than claim live Linux tests;real verification needs corresponding source, fault traces, and frame/PTE counters.

Fork, write, and exec: track object identities



Model: private anonymous page; no concurrent writes, extra mappings, or DMA

In this model, fork creates separate tasks and mms while sharing one private anonymous page F17. If the child writes while both still share it, COW first copies the page to F23. Successful exec replaces the child image; the parent retains its own.

A fault copies only when necessary; an exclusive page may be reused. MAP_SHARED follows different rules, and inherited descriptors may share an open file description.

Clone flags are constrained

Mode	Task/TID	TGID	Mm	Fd table	Existing descriptions
Fork	New	New group	New, private pages may COW	Copied	Shared references
CLONE_VM only	New	Can be a new group	Shared	May be copied	References can share
VM/SIGHAND/THREAD	New	Shared group	Shared	Depends on CLONE_FILES	By table entries

V3.2 rejects THREAD without SIGHAND, and SIGHAND without VM. VM/FS/FILES/SIGHAND alone omit pthread-like thread-group/TGID/signal lifecycle semantics. Parent TID 100/TGID 100 can fork 101/101 or create thread 102/100. Both have independent tasks/stacks despite shared mm.

Real time converted to virtual runtime

For v3.2 fair class, greater weight means slower virtual accumulation and more CPU share. With nice 0 baseline 1024,

$$\Delta v = \Delta t \frac{1024}{w}.$$

Continuously runnable A, nice 0, w=1024 and B, nice 5, w=335 share an ideal 6 ms target period in ratios 1024/1359 and 335/1359, about 4.521/1.479 ms. Both virtual increments are about 4.521 ms. This weighted calculation does not promise exact per-period segments;tick, wakeup, minimum granularity, and other classes change traces.

The fair rbtree's handling of a running entity follows scheduling paths;not every task is always in one tree. RT FIFO/RR can outrank fair and do not compare the same vruntime. Historical predecessors explain design pressures without becoming v3.2's scheduler. Chapter 6's fixed HRTICK/periodic-tick Gantts provide intermediate selections.

How weights determine CPU share and vruntime

CFS weights: actual time and virtual runtime

One CPU; A and B remain runnable for an ideal 6 ms sharing interval

Total weight 1024 + 335 = 1359 $\Delta v = \Delta t \times 1024 / \text{weight}$

Weight share	Actual CPU time	Virtual runtime increase
A: nice 0, weight 1024 1024 / 1359 ≈ 75.35%	6 × 1024 / 1359 ≈ 4.521 ms	6 × 1024 / 1359 ≈ 4.521 ms
B: nice 5, weight 335 335 / 1359 ≈ 24.65%	6 × 335 / 1359 ≈ 1.479 ms	6 × 1024 / 1359 ≈ 4.521 ms

This is an ideal-share calculation, not a trace of every scheduling period.

In this ideal Linux 3.2 model, both tasks remain runnable on one CPU. The lower-weight task B receives less actual CPU time, yet its weighted virtual runtime, calculated before rounding, increases by the same amount as A. Real schedules also depend on ticks, wakeups, and other classes.

Local reentry versus SMP exclusion

A process context and same-CPU IRQ handler both acquire L. If the process holds L without masking that IRQ, the handler interrupts and spins forever because the holder cannot resume. Use the appropriate irq-save protocol and restore prior state rather than unconditionally enable IRQs.

Mechanism	Prevents	Does not prevent
Local preemption disable	Scheduler task switch here	IRQ, other CPUs
Local ordinary IRQ disable	Corresponding local handlers	Other CPUs, NMI
Atomic spinlock	Cooperating contexts on same lock	Nonparticipants, invalid DMA
Irq-save spinlock	Same-lock SMP races and specified local IRQ reentry	Mechanisms outside the protocol

Do not acquire a blocking mutex in nonsleepable paths. Keep spinlock regions short. NMI/softirq/configuration variations need separate analysis rather than one universal priority diagram.

Zones under a fixed x86-32 configuration

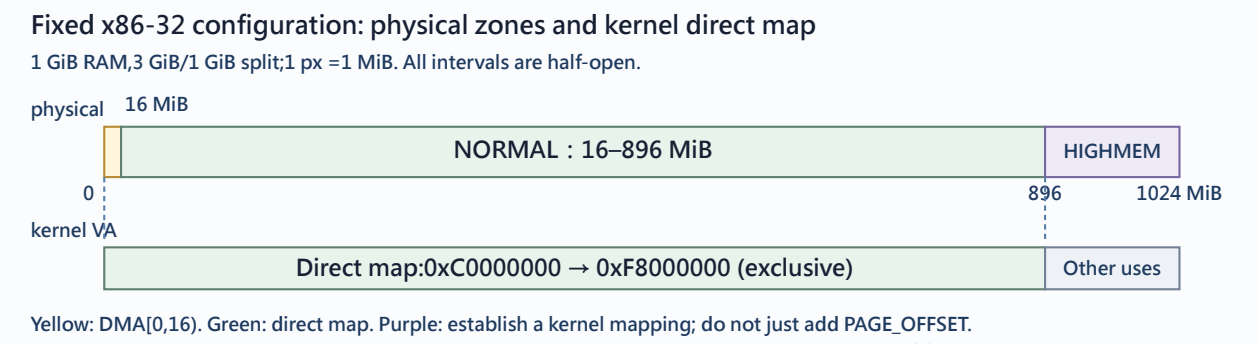
Fix v3.2, x86-32, 4 KiB pages, 3 GiB user/1 GiB kernel split, 896 MiB permanent direct-map coverage, and 1 GiB RAM. These are configuration-specific coverage intervals, not all-free memory.

Physical range	Zone	Constraint
[0,16 MiB)	DMA	Selected legacy ISA limits
[16,896 MiB)	NORMAL	Ordinary permanent direct map
[896 MiB,1 GiB)	HIGHMEM	Requires suitable temporary kernel mapping

PA0x01002000 plus PAGE_OFFSET0xC0000000 gives kernelVA0xC1002000 here. HighmemPA0x39000000 cannot simply be declaredVA0xF9000000, whose region has other uses. Establish a valid mapping. User PTEs can still map highmem:VPN0x40000→PFN0x39000 mapsVA0x40000123 toPA0x39000123.

The physical allocator follows zone/order constraints;buddy allocates power-of-two contiguous pages, slab smaller objects. Device masks, IOMMU, bounce buffers, and DMA API determine DMA addresses;not all DMA is limited to 16 MiB.

Zones under a fixed x86-32 configuration (continued)



ELF bytes and zero-fill

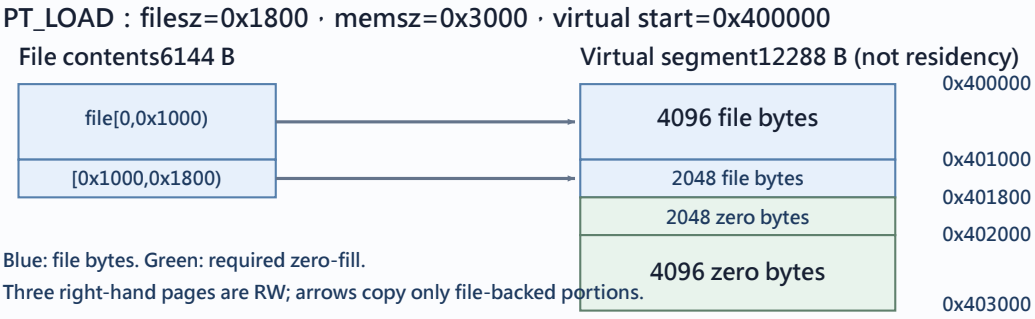
This separate example uses an NX-capable x86-64 ELF permission model, not the preceding x86-32 layout. PT_LOAD has p_offset=0, p_vaddr=0x400000, p_filesz=0x1800, p_memsz=0x3000, RW. Memory beyond filesz must read zero rather than later file garbage.

Virtual page	Contents	Permission
[0x400000,0x401000)	File[0,0x1000)	RW
[0x401000,0x402000)	File[0x1000,0x1800), then 0x800 zero bytes	RW
[0x402000,0x403000)	0x1000 zero bytes	RW

File contents 6144 B plus zero tail 6144 B equal 12288 virtual B. Creating a VMA does not make every page resident. Separate RX segments describe code;private writes use COW, connecting fork and fault handling.

PT_INTERP selects a user-mode dynamic interpreter. Kernel maps program/interpreter, builds initial stack, and enters it;dynamic linker resolves DT_NEEDED/symbols. Kernel need not resolve every function call, and libraries need not always map at one VA.

ELF bytes and zero-fill (continued)



Reclaim preserves backing and live references

Track mapping, backing, dirty state, and reclaim eligibility separately. Reuse cannot occur while valid CPU/device references still require a frame. Clean file-cache data is reconstructable only from valid backing. Dirty data may be the only latest copy;complete the required writeback before marking the covered state clean. Remove unsafe mappings/users under the reclaim protocol and release necessary references before reuse. A cached page can remain referenced while no application currently executes on it. Writeback cleanliness alone is not a full filesystem durability transaction.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

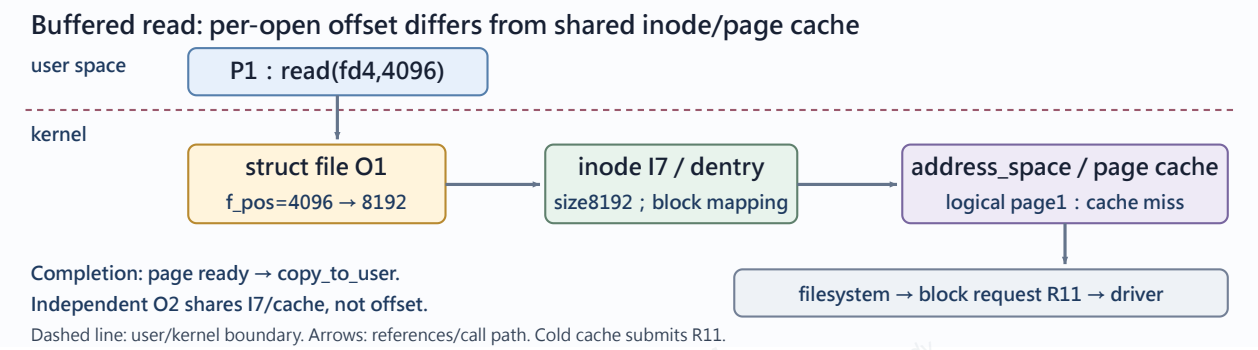
Four VFS objects in one buffered read

Fix v3.2, ordinary ext3 buffered read, size 8192, page 4096, opened P1 fd 4→struct file O1 with f_pos 4096, dentry/inode I7 references. I7.address_space manages shared file pages; superblock identifies the filesystem instance. This example assumes complete successful I/O and copy_to_user, available data, and no concurrent truncate or shared-offset change.

Object	State/references	Incorrect assumption
Superblock	Filesystem instance/operations	Contains per-open offset
Dentry	Parent/name→inode, possibly negative	One unique name per file
Inode	Identity, mode, size, mapping, address_space	One global current offset
Struct file O1	F_pos, flags, operations, path	Every fd necessarily has its own offset

Read 4096 retains O1, validates operation/user buffer, and dispatches file ops. A miss on logical page 1 submits mapped blocks, waits for page readiness, copies to user, and sets O1.f_pos to 8192. Independent O2 can use I7's page cache; dup shares O1. Cache hits save block I/O without eliminating lookup/copy costs.

Four VFS objects in one buffered read (continued)



Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Ext3 block groups and procfs boundaries

Ext3 organizes inodes, data, bitmaps, block groups for locality. Nearby allocation is a policy target, not an invariant that every file stays contiguous. Growth/fragmentation can move later blocks while mappings remain correct.

In the teaching data=ordered model, a 6000 B write dirties two page-cache pages and its metadata transaction publishes newly allocated block mappings. Relevant data must be ordered before the metadata commit that exposes those mappings, avoiding pointers to unready contents. Write success differs from fsync, and metadata atomicity does not make all in-place user writes atomic. Data=writeback and data=journal have different contracts.

Procfs /proc/123/status formats live kernel state rather than necessarily reading persistent blocks. Reused PID 123 can later denote another process;monitoring needs start time/other identity evidence. Separate proc reads do not form a simultaneous system snapshot. A fixed 16-bit PID or always-PID-in-upper-inode-bits rule is not assumed for v3.2.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Three driver paths and terminal states

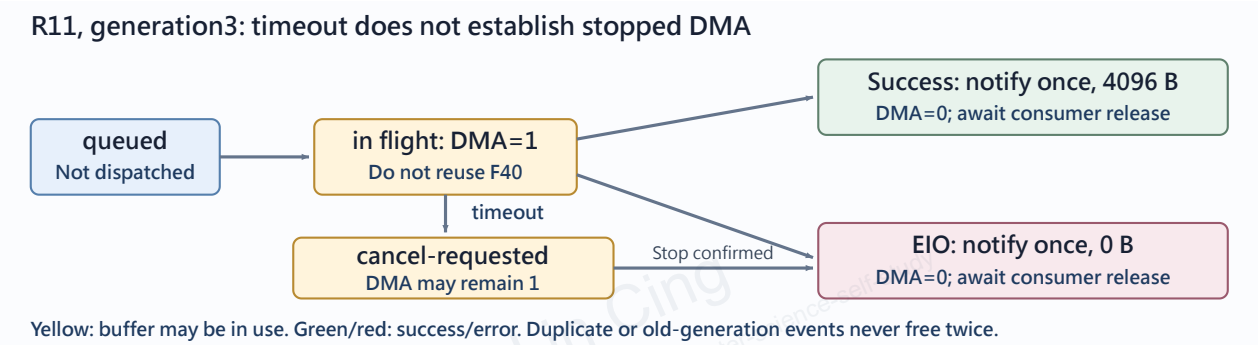
Block devices offer block/sector addressing through filesystem/request layers. Character drivers define file operations without universal seeking. Network devices use sockets/protocols/packets rather than universally seekable device files.

Fix one cold-page synchronous read, resident user buffer, no read-ahead, request R11 over kernel buffer F40. Queue→in-flight device ownership→terminal success/error. Publish results and release mapping according to completion/lifetime. Real layers can merge/split;one page/request is this input model.

Outcome	Hardware evidence	Terminal notifications	Application	Reuse F40
Success	I/O ends, ownership returns	One	4096 B	After copying/references end
EIO	Request stopped/error-completed	One	Error with no prior bytes	After DMA/references end
Timeout, hardware may continue	Stop unproved	Cannot claim success/safe release	Cancellation/reset protocol	Not yet

Three driver paths and terminal states (continued)

An error completion still completes;excluding it from completion counts strands waiters. Timeout is not hardware cessation. Late DMA after free corrupts the next owner's data.



Duplicate completion and reset

Submission identity is (id 11, generation 3). First valid terminal event atomically changes in-flight→terminal and notifies once. Duplicates of that generation are ignored/logged without another notify/free. Reusing id 11 with generation 4 rejects old generation 3 events.

Generation protects software identity, not DMA quiescence. Cancel/reset must establish hardware no longer uses old addresses before synchronization/unmap/reuse. Device-specific late-IRQ/error protocols matter beyond a timeout number.

Synchronous read finishes copying before return. Async user buffers remain valid through the completion contract. Queue state, ownership, and notification count belong in one consistent model.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Pipe EOF requires every writer to close

Parent fd 3→read description R, fd 4→write description W. Fork copies fd tables while retaining R/W objects. A parent consumer and child producer must close unused endpoints.

Event	Parent writer	Child writer	Buffered bytes	Read result
After fork	1	1	0	Block or EAGAIN, not EOF
Child writesABC/closes	1	0	3	ABC available
Consumer drains	1	0	0	Still no EOF
Parent closes last writer	0	0	0	Later read returns 0

Streams preserve byte order, but multiwriter atomicity depends on PIPE_BUF, size, and mode rather than every arbitrary write being indivisible. Shared memory supplies neither queueing nor exclusion;choose locks, semaphores, or an atomic protocol.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Signals, wait queues, and atomic semop

Blocked standard signals can coalesce repeated instances into one pending event, so they are not reliable counters. Real-time signals can queue and sigqueue can carry limited payload. Handlers remain constrained by async-signal-safety.

A waiter that tests done=false before enqueue/sleep can miss a producer's intervening done=true+wake. Establish a wakeable waiting state first, then recheck. If true, do not sleep. Recheck after waking because another waiter can consume the resource. Wake makes runnable rather than immediately executing or permanently satisfying the condition.

System V semop atomically batches operations within a semaphore set. With S0=1, S1=0, the batch S0-=1, S1-=1 must not permanently decrement S0 while waiting for S1. Publish no partial effect before all can complete. Permissions, limits, deletion, and errors remain part of the contract.

Application and wire byte denominators

Fix IPv4/TCP, no options, MTU 1500, no TSO/GSO. Send accepts 6000 B. The MTU permits at most 1460 B of TCP data, but peer MSS, path MTU, scheduling and window can change segmentation. In this specific trace, effective MSS is 1460 B, there is no retransmission, and the sender transmits four full segments followed by one 160 B segment; payloads sum to 6000 B. Five IP/TCP headers add 200 B, L3 bytes total 6200 B, excluding Ethernet, ACK, retransmission.

Sk_buff, TCP send queue, routing, and driver retain different states. Offload can split one large sk_buff into many wire packets, so counts differ. Send acceptance proves neither peer acknowledgment nor application processing nor disk durability.

For 10.1.2.3 with routes /0, 10.0.0.0/8, 10.1.0.0/16, 10.1.2.0/24, the longest match is /24. FIB structures, caches, hooks vary by version/CONFIG;longest-prefix semantics do not imply one universal hash layout.

Queueing, retransmission, and causal evidence

In a fixed trace relative to enqueue0, dispatch2 ms, transmit completion3, ACK13. Queue wait2, dispatch-to-transmit1, post-transmit ACK wait10. These claims follow linked timestamps.

Aggregate queue length20, retransmissions3, device errors1 cannot prove queueing caused a long tail. Loss, reordering, or unrelated device faults may explain counts. Link the same request/flow/window's enqueue, dispatch, completion, drop, ACK evidence, or use a controlled intervention before making a causal claim.

TCP sequence handling restores ordered delivered bytes despite IP loss/reordering. UDP preserves datagram boundaries without delivery guarantees. A TCP upper layer does not make every driver/IP queue lossless or ordered.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Authentication and authorization

PAM composes user-space authentication, account, session policies;kernel resource checks still use credentials. Salt separates stored values without adding password entropy. KDF cost/access protection differ. Old eight-character schemes are not current recommendations.

Credentials include real, effective, saved, filesystem IDs, groups, capabilities. Ordinary file DAC selects owner/group/other then requested bits;ACL, LSM, capability, mount conditions can change the decision. Successful open can create a continuing handle not revoked by later pathname-mode changes.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Root and regular-file DAC

Fix v3.2, a non-immutable regular file using generic_permission, owner 1000, mode 0400, rw mount, no ACL/LSM denial, and successful path traversal. Caller current_user_ns equals inode_userns; each request independently asks for only read, write, or execute. Effective capabilities belong to the corresponding user namespace;exclude other policy.

Caller	Read	Write	Execute	Rule
Fsuid 1000, no DAC capability	Allow	Deny	Deny	Owner bits 100
Fsuid 2000, no capability	Deny	Deny	Deny	Other bits 000
Fsuid 0, CAP_DAC_OVERRIDE	Allow	Allow	Deny	Override read/write;regular execute needs some x bit
Fsuid 0, no relevant capability	Deny	Deny	Deny	Numeric UID alone adds no missing capability
Fsuid 2000, CAP_DAC_READ_SEARCH only	Allow	Deny	Deny	Supplies regular-file read only

Generic_permission first performs ordinary checking, then tests CAP_DAC_OVERRIDE on failure;execute also requires nonzero S_IXUGO. Mode 0401 supplies an x bit, allowing that DAC override, while noexec, format, and other policies still apply. Thus mode 0400 does not force capable-root write denial, and root does not bypass every security layer.

Temporary versus permanent privilege drop

A setuid-root program hasreal1000, effective0, saved0. Seteuid(1000) yields1000/1000/0 and can later restore effective0 under saved-ID rules. It is temporary.

Permanent dropping additionally handles saved IDs, groups, capability sets, and privileged handles. Changing euid alone does not recall opened descriptors. Securebits, user namespaces, and required resources affect implementation;this ID trace is not a complete sandbox recipe.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

One successful read across mechanisms

Continue fd 4→O1→I7, offset 4096, read 4096, cold cache, resident user buffer, no concurrent truncate or shared-offset changes, and complete successful I/O and copy_to_user. Retain fd/validate→cache lookup→filesystem map→R11 block I/O→device completion→ready page→copy_to_user→O1.offset 8192→return 4096. Waiting can switch tasks, unlike an ordinary internal function call.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Errors, partial progress, and lifetimes

If R11 terminates EIO before any copy, read returns error without advancing offset. Terminal notification is still exactly one, and buffer reuse waits for stopped DMA/released references. With earlier copied bytes, an API may return that successful prefix and expose errors later;not every failed read has zero progress. This fixed example explicitly excludes prior bytes.

Fork sharing pages does not share tasks;independent opens sharing inode do not share offsets;send acceptance is not remote receipt;dirty-cache writes are not durable. Identity, authority, lifetime, and completion contracts preserve these distinctions.

References and attribution

Lin Cing, handwritten operating systems notes. Silberschatz et al., Operating System Concepts, 9th ed., Ch. 18. Linux v3.2 fork, memory, module, DAC, ELF. fork(2), signal(7).

Original educational synthesis by Lin Cing, licensed under CC BY 4.0.

https://github.com/xinhi6716/computer-science-self-study